

LEARN ALGORITHMS BY HAND

# NeetCode 150 User Guide

A friendly walkthrough of this practice repo for a complete beginner. What every folder is for, where to work and exactly how a day of practice goes.

**150**

problems

**18**

topics

**1**

rule: no AI solutions

Read section 1 and 3 first, then follow the daily loop in section 5.

## Contents

---

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. What this repo is</b>                  | <b>3</b>  |
| <b>2. A tour of the folders</b>              | <b>4</b>  |
| <b>3. Where to work</b>                      | <b>5</b>  |
| <b>4. The tools</b>                          | <b>6</b>  |
| <b>5. A day of practice (the daily loop)</b> | <b>8</b>  |
| <b>6. Every week (the weekly loop)</b>       | <b>10</b> |
| <b>7. How to approach any problem</b>        | <b>11</b> |
| <b>8. The 18 topics explained</b>            | <b>12</b> |
| <b>9. Getting unstuck</b>                    | <b>20</b> |
| <b>10. When something breaks</b>             | <b>21</b> |

# 1 What this repo is

---

A place to work through 150 well-known coding problems by hand, with small tools that keep you honest.

This repo helps you practise the **NeetCode 150**, a popular list of 150 coding interview problems sorted into 18 topics. It is a place to learn, not a place to store answers. You write every line of every solution yourself. The tools here set up files, run your own tests, track what you have finished and remind you what to revisit. They never write the answer.



## **The one rule: no AI-written solutions, ever**

Not the full answer, not one line, not pseudocode. If you ask the assistant to solve a problem, it will say no and offer a hint instead. This is the whole point of the repo. The rules live in `CLAUDE.md`.

**Why such a strict rule?** The learning happens in the struggle. When you work a problem out yourself, you build the pattern recognition that interviews test. If an assistant hands you the code, you get a working file and learn nothing. A tool that solves the problem for you feels like progress while removing the part that teaches you. So the tools here help you think, they do not think for you.

## 2 A tour of the folders

Green means you type into it. Orange means a tool writes it, so leave it alone.

| Folder or file                                              | What it is                                                                                   | Who owns it       |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------|
| The 18 topic folders<br>arrays–hashing, trees,<br>and so on | Your solution files live here, one file per problem.                                         | YOU EDIT          |
| redo/                                                       | Fresh re-attempts of problems you already solved, so a redo does not overwrite the original. | YOU EDIT          |
| templates/problem-<br>template.py                           | The blank solution file every new problem is copied from.                                    | TOOL COPIES<br>IT |
| helpers/structures.py                                       | Small building blocks for your tests: linked list and tree makers.                           | YOU USE IT        |
| scripts/                                                    | The five tools you run, plus a lib/ of shared code you never run directly.                   | TOOLING           |
| hooks/pre-commit                                            | A safety check git runs for you on every commit.                                             | TOOLING           |
| README.md                                                   | The front page. Its progress list is written by a tool.                                      | TOOL WRITES<br>IT |
| python-gotchas.md                                           | A running list of Python language traps you hit.                                             | YOU ADD TO<br>IT  |
| CLAUDE.md                                                   | The rules for the assistant.                                                                 | REFERENCE         |



### The .gitkeep files

You will see a tiny hidden .gitkeep file in empty folders. It exists only so git can save an otherwise empty folder. You can ignore it.

### 3 Where to work

Read the problem on the website. Write and test it here on your computer. Paste it back to the website to submit.

#### 1. Read on the site

The problem statement and the official grader live on LeetCode. You cannot copy the grader onto your machine, so you read and finally submit there.

#### 2. Solve here

You write the code, your notes and your tests in this repo, in an editor such as VS Code. Your notes, history and the revisit schedule only exist here.

#### 3. Submit on the site

Once your own tests pass, paste your solution into LeetCode and submit to get the official pass.



#### **You do not need the internet to test**

Your solution file has its own tests at the bottom (asserts you write). Running the file checks them. Passing your own tests is not the same as passing LeetCode, because you cannot think of every case the grader will, but it catches most mistakes before you submit.

## 4 The tools

Two kinds. Slash commands you type to the assistant in chat (they start with `/`). Scripts you run in the terminal. Run scripts from the top of the repo.

### Slash commands

#### `/new`

Sets up a new problem file and starts your timer. Says nothing about how to solve it.

#### `/hint`

Gives the next hint, one small step at a time. Never the algorithm.

#### `/review`

Adds edge-case tests and asks about your complexity claims. Never edits your solution.

#### `/done`

Helps you fill your notes, checks the file, updates the README and commits it.

#### `/redo`

Picks the oldest problem due for revisit and has you solve it fresh.

#### `/quiz`

Five quick recall questions from problems you have already solved.

#### `/gotcha`

Saves a Python language trap to `python-gotchas.md` so you know it next time.

#### Tip

Every command restates the no-solutions rule, so it holds even in a long chat.

### Scripts

| Run this                                                                                 | What it does                                                                                    |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>python3 scripts/new_problem.py &lt;category&gt; &lt;number&gt; &lt;slug&gt;</code> | Copies the template into a topic folder, pre-filled, and prints your time limit.                |
| <code>python3 scripts/progress.py</code>                                                 | Rewrites the Progress and Revisit lists in the README. Add <code>--check</code> to only verify. |
| <code>python3 scripts/check_file.py &lt;path&gt;</code>                                  | The honesty check for one file: notes filled in, values valid, tests pass.                      |
| <code>python3 scripts/flashcards.py</code>                                               | Exports your solved problems to a flashcard CSV you can import into Anki.                       |
| <code>python3 scripts/install_hooks.py</code>                                            | Installs the pre-commit safety check. Run this once at the start.                               |

## The pre-commit hook and CI

Git runs a small check right before it saves a commit. If a solution file is not finished (a note left as TODO, or a failing test), the commit is stopped and you see this:

```
FAIL: arrays-hashing/0001-two-sum.py
- field 'Pattern' is still TODO/empty

pre-commit: one or more solution files failed the honesty gate.
Fill in the docstring fields and make the asserts pass, then re-commit.
```



### Do not skip the check

You can bypass it with `git commit --no-verify`, but that defeats the whole point. Fix the listed items and commit again. **CI** runs the same checks on GitHub after you push, as a backstop, so a local quirk cannot hide a broken file.

## 5 A day of practice

One problem, start to finish. This is the loop you repeat every day.

### 1 Pick the next problem and read it on the site

Open the README, find the next empty checkbox and note its number and slug. Open that problem on LeetCode and read it.

### 2 Scaffold the file and start the timer

In chat, type:

```
/new arrays-hashing 1 two-sum
```

The assistant makes the file and tells you your limit: **Easy** 25 minutes, **Medium** / **Hard** 40 minutes. Start a timer.

### 3 Open the file and solve it by hand

The top is a notes block. The bottom is where your code and tests go. Nothing is solved yet:

```
class Solution:
    pass # write the method signature from LeetCode yourself

if __name__ == "__main__":
    # write asserts that call your Solution and check the result
    pass
```

### 4 Run your own tests

An assert calls your method and checks the answer. Run the file from the top of the repo:

```
python3 arrays-hashing/0001-two-sum.py
```

No output means your asserts passed. A failure shows the line.

### 5 Stuck? Ask for one hint

Type `/hint arrays-hashing/0001-two-sum.py`. You get one level. Write down how many hint levels you used.

### 6 Review, then finish

Run `/review <file>` for edge-case tests, then `/done <file>` to fill your notes, check the file, update the README and commit it.

### 7 Submit on LeetCode

Paste your solution into LeetCode and submit for the official pass.



### **When the timer runs out**

Stop. Do not keep grinding. Write down where you got stuck, mark the problem to revisit and, if you want, look at the idea through hints (not the code). A problem you could not finish is exactly the kind you want to see again later. Running out of time is data, not failure.

## 6 Every week

The daily loop moves you forward. The weekly loop makes it stick.



### Spaced repetition is the real trick

You do not remember a pattern because you solved it once. You remember it because you met it again, just as you were about to forget it, and had to pull it back. Revisiting on a schedule is what turns a one-time solve into a pattern you own.

#### /redo

A couple of times a week. It picks the oldest problem due for revisit, restates only the problem (no hint about your old solution), and you solve it fresh in `/redo/`. Then you compare the two and talk about what changed.

#### /quiz

When you finish a topic. Five quick questions drawn only from problems you have already solved, revealed one at a time. Pure recall to keep patterns warm.

### How the revisit date is chosen

| When you finish a problem                      | You will see it again in                             |
|------------------------------------------------|------------------------------------------------------|
| You needed a hint, or you marked it to revisit | <b>3 days</b> (came out hard, so it comes back soon) |
| You solved it clean with no hints              | <b>14 days</b> (came out easy, so it waits longer)   |

Every time you run `progress.py`, any problem whose date has arrived shows up in the Revisit list in the README, oldest first.

## 7 How to approach any problem

---

The same steps, in the same order, every time. With practice it becomes automatic.

- 1 Read it twice, then say it back in your own words**  
If you cannot restate what the input is, what the output is and what makes an answer correct, you do not understand it yet.
- 2 Work two tiny examples by hand**  
On paper, no code. This is where you see the real shape of the problem and the corners the statement hid.
- 3 Say the slow way out loud, and its cost**  
Even "try every pair" counts. Name its time complexity. Now you have a baseline any real answer must beat.
- 4 Find the repeated work, then the structure that removes it**  
Which step does the most repeating? Repeated membership checks want a hash set. Repeated smallest-so-far wants a heap. Sorted data invites two pointers or binary search. This one question is most of algorithm design.
- 5 Now write the code**  
If you did the steps above, the code is you writing down a plan you already have.
- 6 Test the corners**  
Empty input, one element, all duplicates, negatives, the largest allowed size. Most bugs live here.
- 7 State the final time and space cost**  
If it is no better than the slow way, go back to step 4.

## 8 The 18 topics explained

What each idea is, when a problem is hinting at it and the mistakes beginners make. These teach the pattern in general. Working out which problem uses which is the exercise, so no problem names here.



### Arrays & Hashing

An array stores items in a row, so reading the item at a known spot is instant ( $O(1)$ ). A hash table (a dict or set) stores items by a key so that on average adding, finding and removing a key are all instant too. That turns "have I seen this before" from a slow scan into a one-step check.

#### REACH FOR IT WHEN

You keep scanning the same data, counting how often things appear, testing membership fast or removing duplicates.

#### SPEED

Set and dict lookups are  $O(1)$  on average. The same check on a list is a slow  $O(n)$ .

**WATCH OUT** Using a list for repeated membership tests. Changing a dict while looping over it. Forgetting that heavy key collisions can make it slow.



### Two Pointers

Walk through a sequence with two positions instead of one. They often start at opposite ends and move toward each other, or start together and move at different speeds. Moving one pointer can rule out many options at once, usually on sorted data.

#### REACH FOR IT WHEN

The data is sorted and you want a pair or triple with some property, or you compare the front against the back, or rearrange in place.

#### SPEED

One sweep,  $O(n)$  time and  $O(1)$  extra space.

**WATCH OUT** Off-by-one on the stopping condition. Moving the wrong pointer. Forgetting the data has to be sorted for many versions.



## Sliding Window

A range inside a sequence, marked by a left and right edge, that grows and shrinks as you scan. Instead of re-checking a whole range each time, you update a running summary as items enter on the right and leave on the left.

### REACH FOR IT WHEN

You want the longest or shortest run of *adjacent* items with some property.

### SPEED

$O(n)$ : each item enters the window once and leaves once.

**WATCH OUT** Recomputing the window from scratch each step (quietly  $O(n^2)$ ). Shrinking with the wrong condition. Recording the answer at the wrong moment.



## Stack

A pile where you add and remove only from the top: last in, first out, like a stack of plates. Adding, removing and peeking at the top are all  $O(1)$ . Useful when the most recent unfinished thing is the one you need next.

### REACH FOR IT WHEN

Matching or nesting (brackets, tags), needing the most recent item, or undo behaviour.

### SPEED

Push, pop and peek are each  $O(1)$ .

**WATCH OUT** Popping an empty stack. Forgetting items still left at the end. Using a stack when order does not actually matter.



## Binary Search

Find a value in sorted data by cutting the range in half each step: check the middle, throw away the half that cannot hold the answer. A million items take about twenty steps ( $O(\log n)$ ). The same halving works to find the smallest answer that passes a yes/no test.

### REACH FOR IT WHEN

The data is sorted, or you can phrase the goal as "smallest value for which this test passes".

### SPEED

$O(\log n)$  time,  $O(1)$  space.

**WATCH OUT** Endless loops from updating the bounds wrong. Off-by-one on low and high. Confusion over whether an edge is included.



## Linked List

A chain of small boxes (nodes), each holding a value and a pointer to the next box. There is no jumping to the tenth item; you follow the chain ( $O(n)$ ). But if you already hold a node, inserting or removing next to it is  $O(1)$ , just a pointer rewrite.

### REACH FOR IT WHEN

You rearrange links rather than values, reverse or reorder in place, or detect a loop.

### SPEED

Access by position  $O(n)$ , insert or delete given the node  $O(1)$ .

**WATCH OUT** Overwriting a node's next pointer before saving the rest of the chain. Mishandling the ends. Not using a dummy head node.



## Trees

Nodes in a hierarchy: one root at the top, each node holding children below. A binary tree gives each node at most two children. Visit nodes depth first (deep down one branch, then back up) or breadth first (one level at a time). Recursion fits, because each child is itself a smaller tree.

### REACH FOR IT WHEN

The data is hierarchical, you do something at every node, or you care about paths from the root down.

### SPEED

Visiting every node is  $O(n)$ ; the tree height sets recursion depth.

**WATCH OUT** Forgetting the empty-node base case. Mixing up visit orders. Deep recursion on a tall, skewed tree.



## Tries

A tree built from the characters of words. Each node is a letter, and a path from the root spells a prefix. Words sharing a prefix share early nodes, so it stores many strings compactly and answers prefix questions fast. Looking up a word costs  $O(L)$ , its length, no matter how many words are stored.

### REACH FOR IT WHEN

You are asked about prefixes, autocomplete, or many strings share leading characters.

### SPEED

Insert and search are  $O(L)$  in the word length.

**WATCH OUT** Forgetting a marker for "a word ends here". Underestimating the memory it uses.



## Heap / Priority Queue

A structure that always hands you the smallest (or largest) item fast. Python's `heapq` is a min heap: peeking at the smallest is  $O(1)$ , while adding or removing the smallest is  $O(\log n)$ . A priority queue is a line where the most important item leaves first, whatever its arrival order.

### REACH FOR IT WHEN

You repeatedly need the largest or smallest, want the "top k", merge ordered streams, or schedule by priority.

### SPEED

Peek  $O(1)$ , add and remove  $O(\log n)$ .

**WATCH OUT** `heapq` only does minimums (store negatives to fake a max heap). Sorting everything when you only needed the top few.



## Backtracking

Build an answer one choice at a time and abandon a partial answer the moment it cannot work, then undo the last choice and try another. It walks a tree of decisions. It is how you generate every arrangement that fits a rule.

### REACH FOR IT WHEN

The problem asks for all combinations, all permutations, all subsets, or every arrangement that satisfies a rule.

### SPEED

Often exponential, so cutting off dead branches early is what makes it usable.

**WATCH OUT** Not undoing a change after a branch. Storing a reference to a list that keeps changing instead of a copy. Skipping the pruning.



## Graphs

Nodes joined by edges: a general web of connections (trees are a special case). Usually stored as a map from each node to its neighbours. Explore breadth first (spread out level by level, finds the fewest edges) or depth first (follow one path as far as it goes). A grid is often a graph in disguise.

### REACH FOR IT WHEN

Networks, connectivity, reachability, dependencies, or a grid of connected cells.

### SPEED

Both searches cost  $O(V + E)$ , nodes plus edges.

**WATCH OUT** Forgetting a visited set and looping forever. Treating a one-way graph as two-way. Building the neighbour map wrong.



## Advanced Graphs

The named algorithms for harder graph questions. Cheapest path with weighted edges uses Dijkstra (expand the closest unfinished node with a heap). Cheapest wiring to connect everything uses a minimum spanning tree. Ordering steps with dependencies uses topological sort. Tracking which nodes are grouped uses union-find.

### REACH FOR IT WHEN

Edges carry weights, you need the minimum connecting set, you must order dependent steps, or you keep merging groups.

### SPEED

Dijkstra about  $O(E \log V)$ ; union-find nearly  $O(1)$  per operation.

**WATCH OUT** Dijkstra with negative weights (it cannot). Skipping the priority queue. Union-find without path shortening.



## 1-D Dynamic Programming

Solve a big problem by solving smaller versions and saving each answer so it is never computed twice. It fits when the same smaller problems keep reappearing and the best overall answer is built from best answers to the parts. In 1-D, each smaller problem is labelled by one number, so answers live in a simple list.

### REACH FOR IT WHEN

You want a count of ways, or a min/max cost, and each step is a choice that leads to a smaller version of the same problem.

### SPEED

Number of subproblems times the work each does, often  $O(n)$ .

**WATCH OUT** Wrong starting values. Filling the list in an order that needs an answer you have not computed yet. Sizing off by one.



## 2-D Dynamic Programming

The same idea, but each smaller problem now needs two numbers to describe it, so the answers live in a grid. That happens when you compare two sequences at once, move through a grid, or track a range with a start and an end. You fill the grid cell by cell, each from cells already filled.

### REACH FOR IT WHEN

Two sequences compared together, paths across a grid, or a subproblem that needs both a start and an end.

### SPEED

$O(n \times m)$  for a grid of  $n$  by  $m$  cells.

**WATCH OUT** Wrong grid dimensions. Filling in an order that needs cells not ready yet. Mixing up row and column.



## Greedy

Make the choice that looks best right now at every step and never reconsider, hoping local best adds up to global best. Fast and simple, but only correct when the problem has a property that guarantees it. Often you sort first, then make one obvious pick at a time.

### REACH FOR IT WHEN

You maximise or minimise something and there is a natural order to process in, such as by size or end time.

### SPEED

Usually  $O(n \log n)$ , the cost of the sort.

**WATCH OUT** Assuming greedy works without checking. The wrong sort key. Test it against a slow brute force on tiny inputs first.



## Intervals

A range with a start and an end, like a meeting from 2 to 3. Most interval problems sort the ranges (by start, or sometimes by end) and then sweep through once, comparing each to the previous. Sorting brings ranges that interact next to each other.

### REACH FOR IT WHEN

Meetings, bookings, calendars, overlaps, or merging ranges into fewer.

### SPEED

Sort  $O(n \log n)$ , then an  $O(n)$  sweep.

**WATCH OUT** The wrong sort key. An off-by-one on the overlap test (do touching ranges count?). Forgetting to sort.



## Math & Geometry

A grab bag: digits of a number, remainders, greatest common divisors, rotating or scanning a matrix, simple coordinate geometry. No single structure. The common thread is careful handling of arithmetic and indices, where a small slip gives a wrong answer.

### REACH FOR IT WHEN

The problem is about the digits of a number, remainders, or moving around a grid by position.

### SPEED

Varies by task, so reason about each one directly.

**WATCH OUT** Off-by-one when indexing a matrix. Comparing floating point numbers with exact equality. Overflow (rare in Python, a habit worth keeping).

Every integer is a row of bits, and you can act on them directly with AND, OR, XOR, NOT and shifts. Fast, and sometimes solves a problem with no extra memory. XOR is the workhorse: a value XORed with itself is 0 and with 0 is unchanged, so pairs cancel. One integer can also stand in for a small set, one bit per member.

**REACH FOR IT WHEN**

The problem hints at no extra space, asks about odd/even counts, or smells like an XOR cancellation.

**SPEED**

Each bit operation is  $O(1)$ .

**WATCH OUT** The sign bit on negatives. Operator precedence (wrap bit operations in parentheses). Confusing bitwise `&` with logical `and`.

## 9 Getting unstuck

Hints come in levels. You get one level at a time, in order, and never a jump to the answer.

### Level 1: a question

A nudge that does not point anywhere, such as "what would the slow version cost?" or "what are you scanning more than once?"

### Level 2: a concept

Points at the relevant line or names a general idea ("think about what you could precompute"), not the algorithm.

### Level 3: the idea in words

Describes the idea in plain words, no code and no pseudocode. This is the most you get.

Still stuck after level 3? The help continues as more questions, never as a solution.



### Write down how many hint levels you used (0 to 3)

This is not busywork. The hint count feeds the revisit schedule: any problem where you needed a hint comes back sooner, because needing a hint means you have not fully learned it. Being honest here is what keeps the spaced repetition useful.



### Algorithm help is rationed. Python help is free.

Finding the idea is the exercise, so it comes through the hint levels. But the Python *language* is not the exercise: a syntax error, a scoping surprise, a shadowed builtin. Ask about those directly and get a full answer, then save it with `/gotcha` so you know it next time.

## 10 When something breaks

The common snags and what to do about each.

### The hook blocked my commit

A solution file is not finished. Read the bullet list it printed, fill the named notes and make your tests pass, then commit again. Do not use `--no-verify`.

### Imports fail when I run a file

You are running from the wrong folder. Run from the top of the repo: `python3 arrays-hashing/0001-two-sum.py`.

### progress.py says the README is stale

Normal after you finish a problem. Run `python3 scripts/progress.py` (without `--check`) and commit the updated README.

### check\_file lists TODO fields

Your notes are not all filled in. Fill every field (hint count 0 to 3, a real date, revisit yes or no). The easy way is `/done`, which asks you for each.



### It passes here but fails on LeetCode

The official grader found a case yours did not. Read which case failed, add it as a new assert in your file, watch it fail there too, then fix your code until it passes again. Now you have a fixed solution and a permanent test for the case that caught you.



### Start here

New to the repo? Read section 1 and section 3, then follow the seven steps in section 5 for your first problem: `/new arrays-hashing 1 two-sum`.